

# Future Minds 3 — O Desafio

*"FinGuard: Assistente Inteligente de Análise de Reclamações de Clientes"*

## O Problema de Negócio

---

Instituições financeiras recebem diariamente milhares de reclamações de clientes por múltiplos canais: SAC, ouvidoria, redes sociais, Procon e Banco Central. Essas reclamações chegam em texto livre, sem padronização, com linguagem informal, erros de digitação, palavras impróprias e diferentes níveis de urgência, desde uma insatisfação com a taxa cobrada até denúncias de fraude ou uso indevido de dados pessoais.

Hoje, o processo típico funciona assim: uma equipe de analistas lê manualmente cada reclamação, classifica em categorias pré-definidas, identifica o produto envolvido (cartão de crédito, conta corrente, empréstimo, investimentos, seguros), avalia o grau de criticidade e encaminha para a área responsável. Reclamações com indícios de fraude ou violação regulatória precisam ser escaladas para compliance. Todo o processo é lento, inconsistente entre analistas e propenso a erros, especialmente nos picos de volume, como períodos pós-lançamento de produtos ou crises de imagem.

O desafio propõe que os participantes construam o **FinGuard** — um sistema inteligente que automatiza e aprimora esse fluxo de análise, desde a triagem inicial até a geração de relatórios acionáveis para a gestão.

### Por que esse problema?

Este desafio foi escolhido porque ele é simultaneamente **realista, relevante e escalável em complexidade técnica**:

**Realista:** análise de reclamações é um processo que existe em toda instituição financeira e está diretamente ligado à experiência do cliente e à conformidade regulatória. Bancos como o ING já utilizam IA agêntica para reduzir em até 90% o tempo de processos de onboarding e compliance. A análise inteligente de reclamações é o próximo passo natural.

**Relevante:** tanto para o Itaú quanto para a Zup, que opera com clientes do setor financeiro, a capacidade de construir soluções de IA que lidam com dados não-estruturados e sensíveis é uma competência estratégica.

**Escalável:** o mesmo problema pode ser resolvido com um chatbot simples (nível básico), com um sistema multi-agente orquestrado (nível intermediário) ou com uma arquitetura completa com guardrails, documentação e consciência de custos (nível avançado)

# Notas de compromisso

**1. Dados e Privacidade** Todos os participantes devem **utilizar exclusivamente o dataset fornecido pela equipe do Future Minds**. É proibido incorporar dados reais de clientes, colaboradores ou qualquer informação oriunda de ambientes corporativos de produção, mesmo para fins de teste ou enriquecimento.

**2. Ambientes de Execução** A implementação deve ocorrer exclusivamente nos ambientes designados para o desafio. Fica vedado o uso de ambientes corporativos de produção, homologação ou desenvolvimento institucional, incluindo buckets, endpoints, bancos de dados ou qualquer recurso vinculado a operações reais.

**3. Desprovisionamento de Recursos** Todos os recursos cloud que gerem custo (endpoints SageMaker, instâncias, buckets S3 criados para o desafio, guardrails configurados, entre outros) devem ser **desprovisionados imediatamente após o término do evento**. A responsabilidade pelo desprovisionamento é da equipe participante, e custos gerados por negligência após o prazo de encerramento serão de responsabilidade do participante ou de sua gestão direta.

# O Dataset

Será fornecido aos participantes um dataset simulado (CSV) contendo reclamações fictícias de clientes, com os seguintes campos:

| Campo            | Descrição                            | Exemplo                                                           |
|------------------|--------------------------------------|-------------------------------------------------------------------|
| id               | Identificador único                  | REC-2026-00142                                                    |
| data_reclamacao  | Data do registro                     | 2026-01-15                                                        |
| canal            | Canal de origem                      | SAC, Ouvidoria, Banco Central, Redes Sociais                      |
| texto_reclamacao | Texto livre da reclamação            | "Fui cobrado duas vezes na fatura do cartão e ninguém resolve..." |
| produto          | Produto envolvido (pode estar vazio) | Cartão de Crédito, Conta Corrente, Empréstimo, etc.               |
| status           | Status atual                         | Aberta, Em análise, Resolvida                                     |

O dataset conterá aproximadamente 500 reclamações com variações de tom (neutro, irritado, formal, informal), complexidade (simples reclamação vs. indício de fraude) e completude (algumas terão produto preenchido, outras não). *Isso força os participantes a lidar com dados imperfeitos, como acontece no mundo real.*

**Nota para ZUnity e Itaú:** o dataset será o mesmo, garantindo paridade de condições. As diferenças estão apenas nas ferramentas e instruções de acesso.

# Nível 1 — Básico: "O Classificador Inteligente"

---

## Objetivo

Construir uma aplicação que receba uma reclamação de cliente (texto livre) e retorne automaticamente uma análise estruturada.

## O que deve ser entregue

Uma aplicação funcional (pode ser CLI, API ou interface web simples) que, dado o texto de uma reclamação, retorne:

- **Categoria da reclamação** — classificação em uma das categorias: Cobrança Indevida, Atendimento, Fraude/Segurança, Produto/Serviço, Cancelamento, Outros
- **Produto identificado** — qual produto bancário está envolvido (Cartão de Crédito, Conta Corrente, Empréstimo, Investimentos, Seguros, Não Identificado)
- **Sentimento** — Positivo, Neutro, Negativo, Crítico
- **Nível de urgência** — Baixa, Média, Alta, Crítica
- **Resumo** — um resumo de 2-3 linhas da reclamação em linguagem padronizada

## Exemplo de entrada/saída

### Entrada:

*"Já é a terceira vez que ligo pedindo o estorno de uma cobrança no meu cartão que eu não fiz. Ninguém resolve nada. Vou procurar o Banco Central se não resolverem até sexta."*

### Saída esperada:

```
JSON
{
  "categoria": "Cobrança Indevida",
  "produto": "Cartão de Crédito",
  "sentimento": "Crítico",
  "urgencia": "Alta",
  "resumo": "Cliente relata cobrança não reconhecida no cartão de crédito, com três tentativas de contato sem resolução. Ameaça escalar para Banco Central."
}
```

## Ferramentas esperadas

- **GitHub Copilot** ou **Amazon Q** para acelerar o desenvolvimento da aplicação
- **Modelo de inferência (LLMs)** para chamada ao modelo que fará a classificação, podem ser utilizados via API ou interface WEB
- **RAG** (local ou por meio de API) para acesso a informações contextuais do documento de **Política Interna**
- Linguagem livre (Python recomendado)

## Competências avaliadas

- Capacidade de usar assistente de código para produtividade
- Construção de prompt eficaz para a tarefa de classificação
- Qualidade e consistência das classificações ao processar múltiplas reclamações do dataset
- Aplicação funcional e demonstrável
- Construção de Agents

## Validação de resultado

- Documento .html com gráfico dos resultados das análises
- Documento .json ou .csv com os resultados do processo de cada reclamação
- Nas mensagens do resumo as palavras impróprias devem ser ofuscadas/removidas

## Requisitos não funcionais

- A solução pode ser executada localmente, sem persistir dados ou resultados gerados em serviços de armazenamento em nuvem (por exemplo, AWS S3).
- A solução pode ser implementada em ambiente Microsoft Copilot Studio, utilizando os recursos de orquestração, Knowledge Source e Guardrails (Bedrock) / Tópicos (MS Copilot);

# Nível 2 — Intermediário: "O Orquestrador de Análise"

---

## Objetivo

Evoluir a solução do nível básico para um **sistema multi-agente orquestrado** que não apenas classifica, mas analisa, enriquece e gera relatórios a partir das reclamações.

## O que deve ser entregue

Um sistema que utiliza **agentes** com responsabilidades distintas, orquestrados por um framework (LangGraph recomendado): **Arquitetura de Múltiplos Componentes**

Este fluxo **pode ser composto por várias etapas especializadas**, cada uma com uma função distinta. A seguir, demonstramos 3 destas etapas, como exemplo, que podem representar o mínimo necessário para construção do fluxo do desafio.

### 1. Recepção e Estruturação (Ponto de Entrada):

- **Função:** Receber a reclamação original (bruta).
- **Saída:** Uma análise estruturada inicial contendo categoria, produto, sentimento associado, nível de urgência e um resumo da queixa.

### 2. Análise de Risco e Conformidade:

- **Função:** Avaliar a saída estruturada da etapa anterior em busca de potenciais indicativos de risco ou não-conformidade.
- **Pontos de Verificação:**
  - Fraude ou transação não autorizada.
  - Violação de regulamentos (ex: LGPD, sigilo bancário).
  - Risco reputacional (menção a imprensa, redes sociais ou órgãos reguladores).
  - Necessidade de escalção imediata.
- **Saída:** Um parecer detalhado com o Nível de Risco (Baixo, Médio, Alto ou Crítico) e a respectiva justificativa.

### 3. Geração de Relatório Gerencial:

- **Função:** Consolidar as informações geradas pelas etapas anteriores.
- **Saída:** Um relatório gerencial em formato estruturado (JSON, Markdown ou HTML), incluindo:
  - Um *Dashboard* resumido (total de reclamações processadas, distribuição por categoria, produto e urgência).
  - Uma lista destacada das reclamações classificadas como críticas, juntamente com seu parecer de risco.
  - Recomendações de ações específicas para a equipe de gestão.

## Estrutura mínima do grafo

None



Os participantes são encorajados a ir além da estrutura mínima, por exemplo, adicionando fluxos condicionais (se urgência for crítica, pular direto para classificação), paralelismo (processar múltiplas reclamações simultaneamente) ou loops de validação.

## Ferramentas esperadas

- O que for necessário do nível 1 mais:
  - **LangGraph** framework ou equivalente para orquestração (se executado via código)
  - **SDK do Iara** ou **Boto3** para integração (se executado via código)
  - Ferramentas de orquestração (Bedrock Agents / MS Copilot Agents)

## Competências avaliadas

- Projeto e implementação de arquitetura multi-agente
- Separação clara de responsabilidades entre agentes
- Qualidade da orquestração (o fluxo faz sentido? os agentes se comunicam bem?)
- Qualidade e utilidade do relatório gerado
- Uso combinado de múltiplas ferramentas de IA

## Requisitos não funcionais

- A solução pode ser executada localmente, sem persistir dados em serviços de armazenamento em nuvem (como AWS S3);
- O sistema deve registrar logs das execuções dos agentes (entrada, saída e tempo de resposta de cada agente)
- O fluxo de orquestração deve ser rastreável, deve ser possível identificar em qual agente uma reclamação está sendo processada;
- O relatório final deve ser gerado em arquivo (JSON, Markdown ou HTML) ao término do processamento;

# Nível 3 — Avançado: "O Arquiteto da Solução"

---

## Objetivo

Construir a solução completa com **segurança, governança, documentação e consciência de custos**, como se fosse para produção real.

## O que deve ser entregue

Tudo do nível intermediário, mais:

### 1. Guardrails de Proteção (obrigatório usar Bedrock Guardrails)

- Implementar um nó de guardrail no início do grafo que valide o input antes de processá-lo
- O guardrail deve bloquear:
  - Tentativas de prompt injection (Jailbreak / Roubo de identidade / Extração do system prompt / Exposição de histórico e dados / Exfiltração de dados)
  - Conteúdo que não seja uma reclamação válida (ex: alguém tentando usar o sistema para outros fins)
  - Mensagens de ameaças as pessoas e instituições (que claramente não são uma reclamação)
- Se o guardrail bloquear, o sistema deve retornar uma resposta educada explicando que não pode processar aquela entrada

### 2. Guardrails de Saída

- Validar que as respostas dos agentes não contêm dados sensíveis do cliente (CPF, números de conta) nas saídas de relatório
- Garantir que o tom das respostas é profissional e neutro

### 3. ADR (Architectural Decision Record)

Documentar a arquitetura em um ADR navegável (página HTML) contendo:

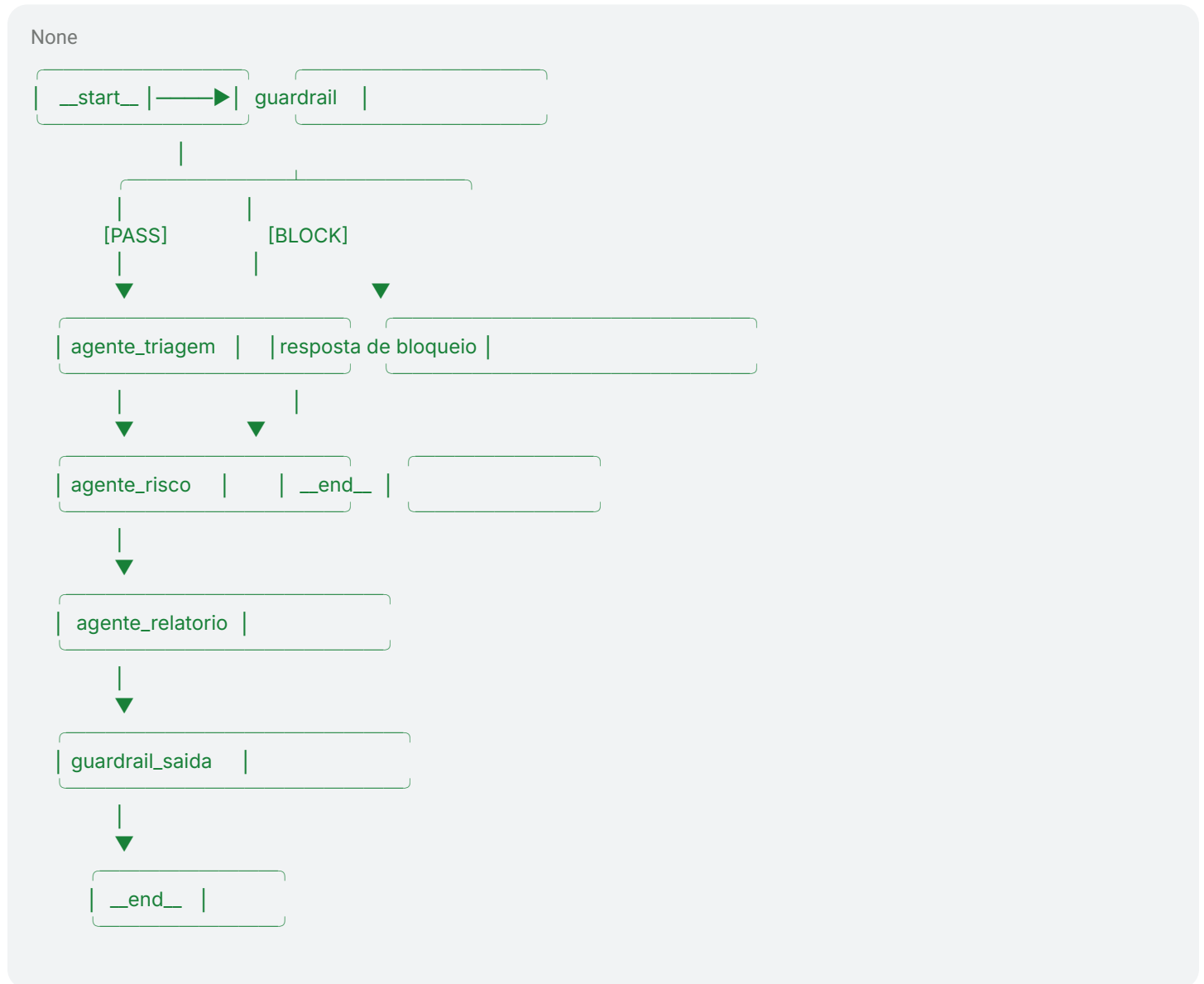
- Contexto do problema
- Opções de arquitetura consideradas (pelo menos 2 alternativas)
- Decisão final e justificativa
- Consequências (trade-offs)
- Análise de custos estimada (quais modelos foram escolhidos e por quê)
- Recomendações de segurança

### 4. Justificativa de Custos

O participante deve demonstrar consciência de custos: (Estas justificativas fazem parte da avaliação.)

- Por que escolheu determinado modelo?
- Comparou o custo de preço por token?
- Usou modelo menor para tarefas simples e modelo maior para tarefas complexas?

## Estrutura mínima do grafo



## Ferramentas esperadas

- O que for necessário do nível 2 mais:
  - Bedrock Guardrails / MS Copilot Tópicos) obrigatório
- Knowledge Source **RAG** para acesso a informações contextuais do documento de **Política Interna**
- **Ferramentas** para orquestração
- Combinação estratégica de modelos (ex: modelo leve para triagem, modelo robusto para análise de risco)

## Competências avaliadas

Todas do nível intermediário, mais:

- Implementação funcional de guardrails (entrada e saída)
- Qualidade e completude do ADR
- Justificativa técnica e financeira das escolhas
- Visão de produção: a solução está pronta para ser apresentada a um stakeholder?

## Requisitos não funcionais

- Todos os requisitos do nível 2, mais:
  - Nenhum dado sensível do cliente (CPF, número de conta, dados pessoais) deve aparecer nas saídas dos agentes ou no relatório final;
  - O guardrail/tópicos de entrada deve ser o primeiro nó do grafo, nenhuma reclamação pode ser processada sem passar por ele;
  - As respostas de bloqueio devem ser educadas, em português e sem expor detalhes internos do sistema;
  - O ADR deve ser entregue como arquivo HTML navegável junto ao repositório;

# Nível 4 — Extra: "O Cientista de Dados"

---

**▲ Nível opcional e sem penalização. Recomendado para participantes com background em Machine Learning. Não utilizar o SageMaker não reduz a pontuação, mas utilizá-lo bem garante uma bonificação significativa na avaliação final.**

---

## Objetivo

Ir além da IA generativa e aplicar técnicas de Machine Learning para extrair padrões e inteligência do dataset de reclamações de forma que LLMs sozinhas não conseguiriam. O participante deve propor, justificar e, se possível, implementar uma abordagem de ML não supervisionada utilizando o **Amazon SageMaker**, integrada ao pipeline do FinGuard.

## O que pode ser entregue

Há duas formas de pontuar neste nível extra, com pesos diferentes:

**Implementação funcional** — solução rodando com SageMaker integrado ao pipeline:

---

- Pipeline de geração de embeddings a partir do texto das reclamações
- Treino de modelo não supervisionado no SageMaker (ex: K-means, LDA ou algoritmo equivalente) para agrupamento das reclamações por similaridade
- Deploy do modelo como endpoint SageMaker
- Agente LLM consumindo os clusters gerados para interpretar e nomear os grupos descobertos
- Análise crítica:
  - Os padrões descobertos coincidem com as categorias pré-definidas?
  - O que divergiu e por quê?

**Projeto arquitetural** — para quem não tiver tempo de implementar:

---

- Diagrama e descrição da arquitetura proposta
- Justificativa da escolha do algoritmo e do número de clusters
- Análise de custos estimada
- Explicação de como o modelo se integraria ao pipeline existente

## Ferramentas esperadas

- **Amazon SageMaker** — treino, avaliação e deploy do modelo
  - **S3** - para eventuais armazenamentos de arquivos e resultados das análises
  - **Bedrock Embeddings** — geração de vetores semânticos a partir do texto
  - **LangGraph** — integração do endpoint como etapa do pipeline de agentes
  - **Boto3** — integração via SDK
  - **GitHub Copilot** ou **Amazon Q** para acelerar o desenvolvimento da aplicação
-

## Competências avaliadas

- Capacidade de identificar onde ML agrega valor além da IA generativa
- Qualidade do pipeline de preparação dos dados
- Treino, avaliação e deploy no SageMaker
- Integração coesa do modelo ao pipeline existente
- Justificativa técnica e financeira da abordagem escolhida
- Conhecimento de algoritmos de ML supervisionado e/ou não supervisionado
- Capacidade de avaliar a qualidade do modelo (métricas, validação, interpretação dos resultados)
- Visão crítica sobre os limites de cada abordagem: quando usar ML clássico vs. LLM

## Requisitos não funcionais

- Todos os requisitos do nível 3, mais:
- O endpoint do SageMaker deve ser removido (delete) ao final do desafio para evitar custos contínuos;
- O pipeline de embeddings deve processar o dataset completo em menos de 10 minutos;
- O número de clusters deve ser justificado com pelo menos uma métrica de avaliação (ex: Elbow Method, Silhouette Score);
- O modelo treinado deve ser versionado e armazenado no S3 com nome e data identificáveis;

# Critérios de Avaliação (todos os níveis)

| Critério                     | Peso | Descrição                                                       |
|------------------------------|------|-----------------------------------------------------------------|
| Funcionalidade               | 30%  | A solução funciona? Faz o que propõe? Demonstração ao vivo.     |
| Uso de Ferramentas de IA     | 20%  | Usou GitHub Copilot? Bedrock/Lara? Soube escolher e justificar? |
| Arquitetura e Design         | 20%  | Separação de responsabilidades, orquestração, clareza do fluxo. |
| Segurança e Governança       | 15%  | Guardrails, proteção de dados, consciência de riscos.           |
| Apresentação e Justificativa | 15%  | Pitch claro, ADR (quando aplicável), justificativa de custos.   |

**Pergunta obrigatória da banca:** "Quais ferramentas de IA você utilizou e como elas contribuíram para a sua solução?"

**Critério bônus:** Participantes que demonstrarem otimização de custos (ex: uso inteligente de modelos menores para tarefas simples) recebem pontuação extra.

# Logística e Regras

- **Formato:** Challenge presencial/remoto (4h de execução)
- **Dataset:** Fornecido no início do challenge (mesmo para todos)
- **Equipes:** Duplas ou grupos (a definir)
- **Linguagem:** Livre (Python recomendado)
- **Entrega:** Será uma **Apresentação** em 10min sendo pitch de 5 minutos + perguntas da banca (5 min)

## Diferenças por público

| Item           | Itaú                                          | Zup                                       |
|----------------|-----------------------------------------------|-------------------------------------------|
| GitHub Copilot | Solicitar acesso prévio (aprovação do gestor) | Licença a ser provisionada                |
| Bedrock        | Disponível via conta DevOps                   | Conta AWS própria (custo estimado)        |
| Iara GenAI     | Disponível via SDK interno                    | Não disponível — usar Bedrock diretamente |
| Execução       | Local (Boto3 + SDK Iara)                      | Local (Boto3)                             |